Here is the manager protocol:

```
object Manager {

  sealed trait Command
  final case class Delegate(texts: List[String]) extends Command
  private case class Report(description: String) extends Command

  ...
}
```

The worker protocol is also slightly different from the previous example. The `Parse` message does not need to contain the text to be printed; now the text is passed to the worker as an input parameter when it is created. `Done` also does not contain a reference to the text to be parsed; the `Report` used in the `ask` contains that reference. Here is the worker protocol:

```
object Worker {

  sealed trait Command
  case class Parse(replyTo: ActorRef[Worker.Response]) extends Command  ❶

  sealed trait Response
  case object Done extends Response                                     ❷

  ...
}
```

❶ Command that the actor can receive

❷ Response that the actor can send

After defining both sides of the communication, let's see how `ask` works.

3.2.4 Asking